

將採用 Angular 的生成式 AI 網頁應用程式，自動從版本管控系統部署至 Cloud Run

來源：<https://codelabs.developers.google.com/codelabs/deploy-from-github/gen-ai-angular?hl=zh-tw>

步驟數：16

目錄

1. [總覽](#)
2. [必要條件](#)
3. [專案設定](#)
4. [開啟 Cloud Shell 編輯器](#)
5. [啟用 API](#)
6. [設定 Git](#)
7. [編寫程式碼](#)
8. [建立存放區](#)
9. [分享代碼](#)
10. [設定自動部署](#)
11. [變更驗證碼](#)
12. [重新部署](#)
13. [\(選用\) 稽核 Vertex AI 使用情形](#)
14. [\(選用\) 提高 AI 工作負載的可觀測性](#)
15. [\(選用\) 清除](#)
16. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 總覽

首次部署網頁應用程式時可能會感到困難重重，即使是首次部署後，如果這個程序太過繁瑣，您可能會避免部署新版應用程式。透過持續部署，您可以輕鬆自動部署應用程式的變更。

在本實驗室中，您將編寫網路應用程式，並設定 [Cloud Run](#)，在應用程式的原始碼變更時自動部署應用程式。然後修改應用程式，再次部署。

學習目標

- 使用 Cloud Shell 編輯器編寫網頁應用程式
 - 在 GitHub 中儲存應用程式程式碼
 - 自動將應用程式部署至 Cloud Run
 - 使用 Vertex AI 在應用程式中加入生成式 AI
-

步驟 2 · 約 0 分鐘

2. 必要條件

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。
 - 請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。
 2. 如果您還沒有 GitHub 帳戶，必須先[建立帳戶](#)
 - 如果有 GitHub 帳戶，可以直接使用。GitHub 比較可能將新帳戶視為垃圾內容帳戶並封鎖。
 - [設定雙重驗證的 GitHub 帳戶](#)，較不容易被當成垃圾內容帳戶。
-

3. 專案設定

1. 登入 [Google Cloud 控制台](#)。
 2. 在 Cloud 控制台中[啟用帳單](#)。
 - 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
 - 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
 - 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。
 - 參加「Gen AI for Devs」活動？您可能可以獲得 [\\$1 美元抵免額](#)。
 3. [建立新專案](#)，或選擇重複使用現有專案。
 4. 確認 Cloud Billing 的「我的專案」
 - 如果新專案的「Billing account」欄顯示 `Billing is disabled`，請按照下列步驟操作：
 1. 按一下「Actions」欄中的三點圖示
 2. 按一下「變更帳單」
 3. 選取要使用的帳單帳戶
 - 如果您參加的是「生成式 AI 開發人員活動」，帳戶名稱可能為「Google Cloud Platform 試用帳單帳戶」
-

步驟 4 · 約 0 分鐘

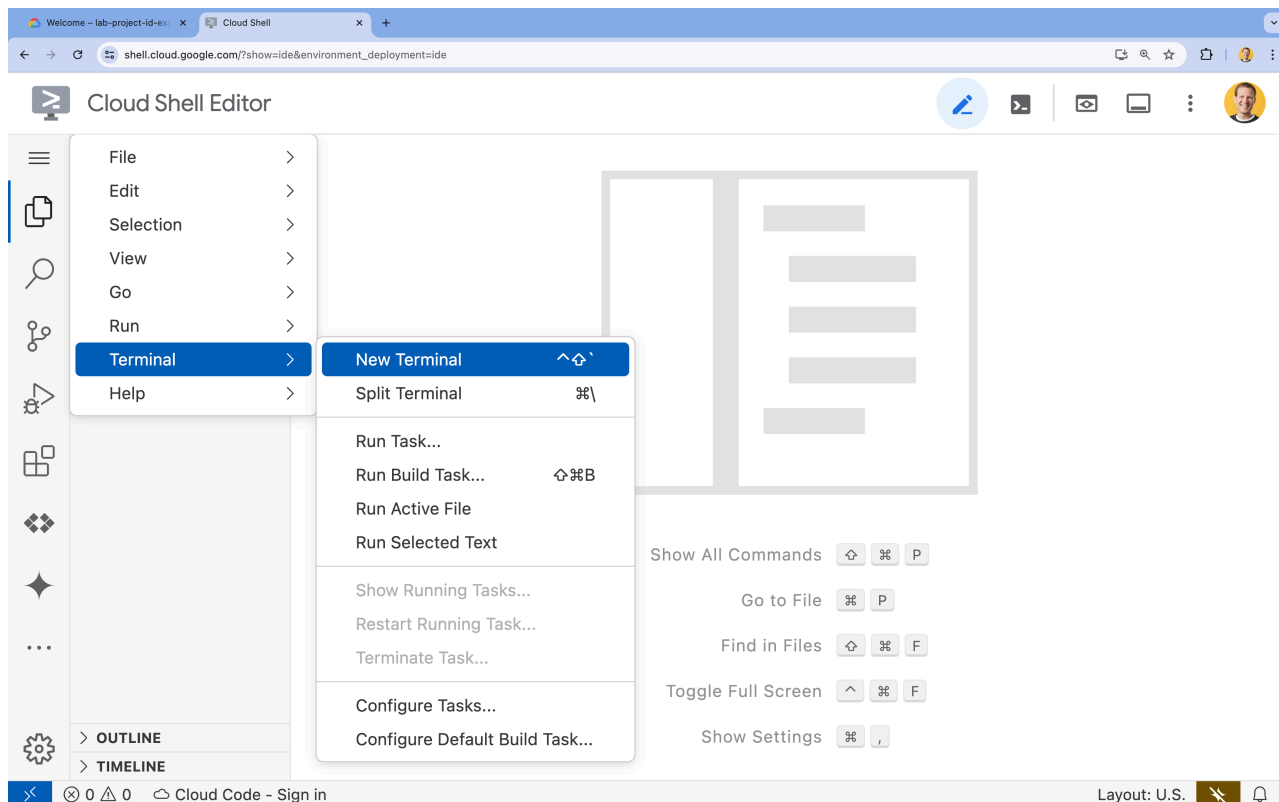
4. 開啟 Cloud Shell 編輯器

1. 前往 [Cloud Shell 編輯器](#)
2. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下漢堡選單



- 按一下「終端機」。
- 按一下「New Terminal」(新增終端機)



3. 在終端機中，使用下列指令設定專案：

- 格式：

```
gcloud config set project [PROJECT_ID]
```

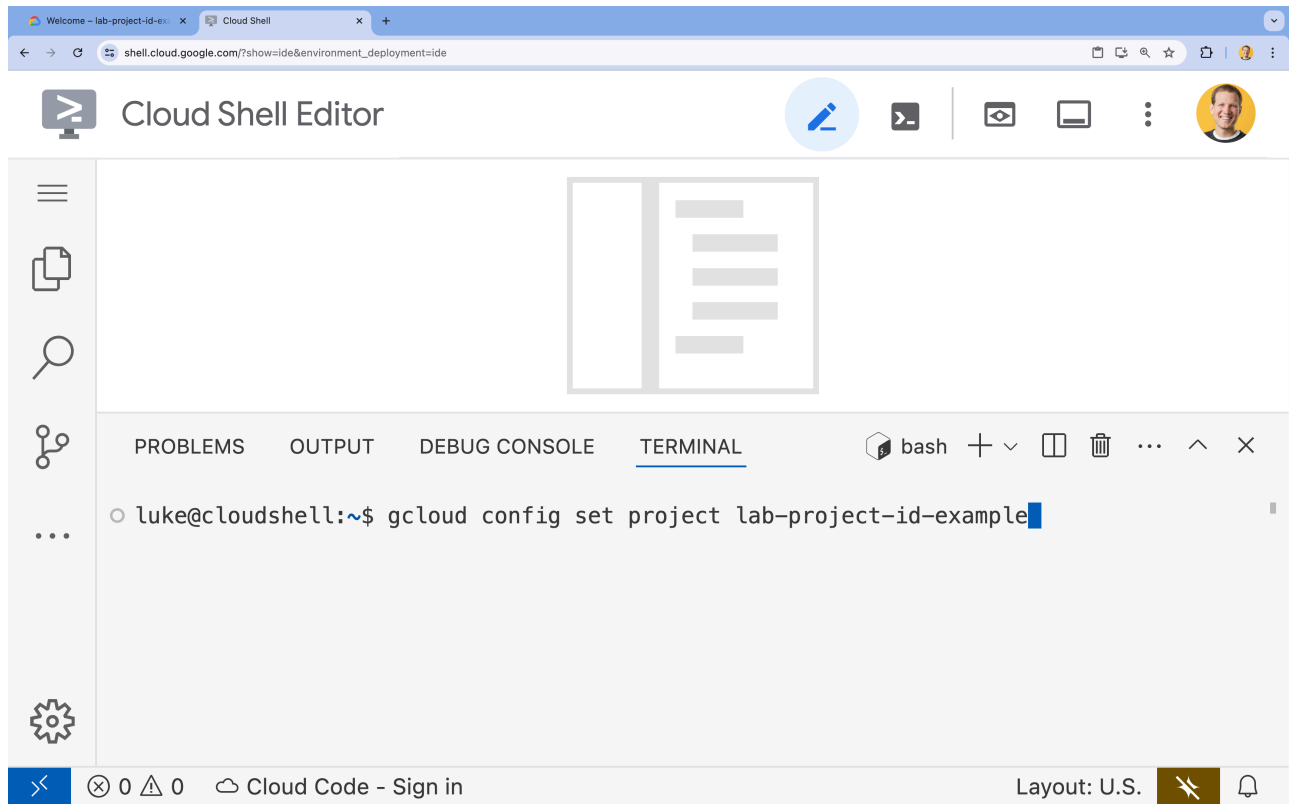
- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，請按照下列步驟操作：

- 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



4. 如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/N)?` 提示，可能是輸入的專案 ID 有誤。按下 `N` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 5 · 約 0 分鐘

5. 啟用 API

在終端機中啟用 API：

```
gcloud services enable \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  aiplatform.googleapis.com
```

這個指令可能需要幾分鐘才能完成，但最終應該會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

步驟 6 · 約 0 分鐘

6. 設定 Git

1. 設定全域 git 使用者電子郵件：

```
git config --global user.email "you@example.com"
```

2. 設定全域 git 使用者名稱：

```
git config --global user.name "Your Name"
```

3. 將全域 git 預設分支版本設為 `main`：

```
git config --global init.defaultBranch main
```

步驟 7 · 約 0 分鐘

7. 編寫程式碼

如要使用 Node.js 編寫應用程式，請按照下列步驟操作：

1. 前往主目錄：

```
cd ~
```

2. 建立新的 `codelab-genai` Angular 應用程式：

```
npx @angular/cli new codelab-genai \
  --minimal \
  --inline-template \
  --inline-style \
  --ssr \
  --defaults
```

3. 如果系統要求安裝 `@angular/cli`，請按下 `Enter` 繼續：

```
Need to install the following packages:
@angular/cli@18.2.4
Ok to proceed? (y)
```

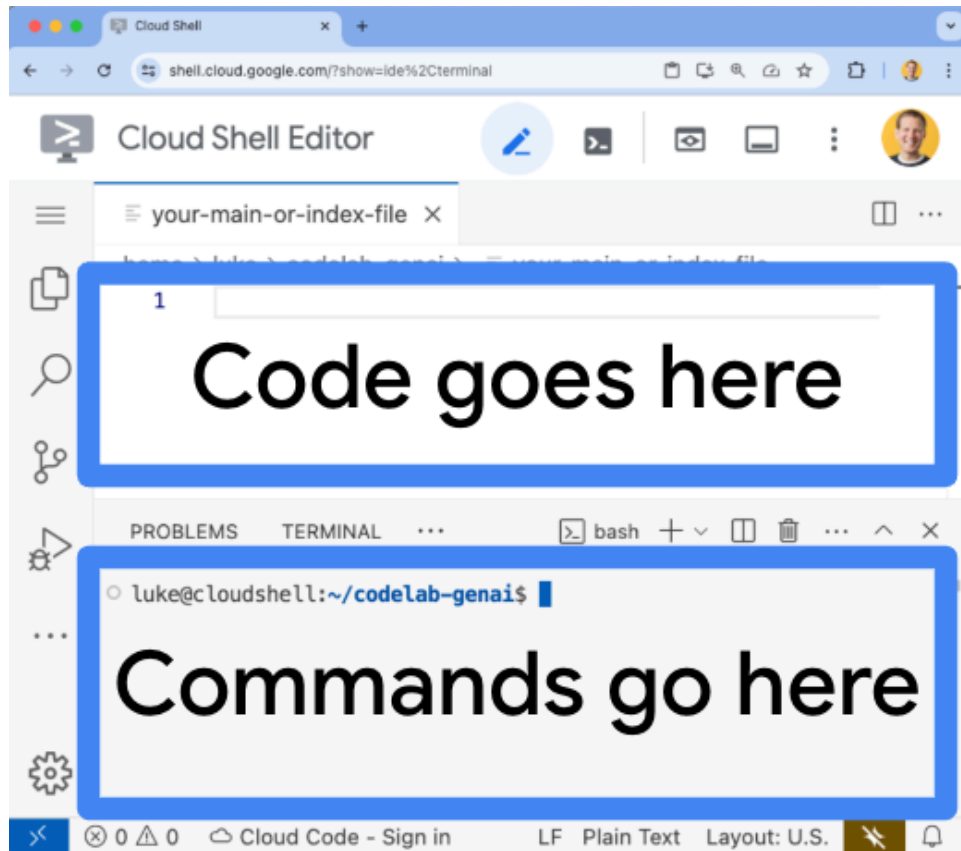
4. 請前往 `codelab-genai` 目錄：

```
cd codelab-genai
```

5. 在 Cloud Shell 編輯器中開啟 `app.component.ts` 檔案：

```
cloudshell edit src/app/app.component.ts
```

畫面頂端現在應該會顯示空白檔案。您可以在這裡編輯這個 `app.component.ts` 檔案。



6. 複製下列程式碼，並貼到開啟的 `app.component.ts` 檔案中：

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  template: `
    <h1>Hello world!</h1>

    <router-outlet />
  `,
  styles: [],
})
export class AppComponent { }
```

幾秒後，Cloud Shell 編輯器會自動儲存程式碼。這段程式碼會以「Hello world!」問候語回應 HTTP 要求。

應用程式的初始程式碼已完成，可以儲存在版本控制系統中。

步驟 8 · 約 0 分鐘

8. 建立存放區

1. 返回畫面底部的 Cloud Shell 終端機。
2. 確認您仍位於正確的目錄：

```
cd ~/codelab-genai
```

3. 初始化 Git 存放區

```
git init -b main
```

4. 登入 GitHub CLI

```
gh auth login
```

按下 `Enter` 鍵接受預設選項，然後按照 GitHub CLI 工具中的指示操作，包括：

1. 你想登入哪個帳戶？ `GitHub.com`
2. 您偏好使用哪種通訊協定，在這個主機上執行 **Git** 作業？ `HTTPS`
3. 是否要使用 **GitHub** 憑證驗證 **Git**？ `y` (如果沒有顯示，請略過此步驟)。
4. 您要如何驗證 **GitHub CLI**？ `Login with a web browser`
5. 複製一次性驗證碼
6. 開啟 <https://github.com/login/device>
7. 貼上一次性代碼
8. 按一下「Authorize github」(授權 GitHub)
9. 完成登入

5. 執行下列指令，確認您已登入：

```
gh api user -q ".login"
```

如已成功登入，輸出內容應為您的 GitHub 使用者名稱。

6. 建立 `GITHUB_USERNAME` 變數

```
GITHUB_USERNAME=$(gh api user -q ".login")
```

7. 執行下列指令，確認已建立環境變數：

```
echo ${GITHUB_USERNAME}
```

如已成功建立變數，輸出內容應為您的 GitHub 使用者名稱。

8. 建立空的 GitHub 存放區，並命名為 `codelab-genai`：

```
gh repo create codelab-genai --private
```

如果收到下列錯誤訊息：

```
GraphQL: Name already exists on this account (createRepository)
```

這樣您就已經有名為「`codelab-genai`」的存放區。如要繼續按照本教學課程操作，有兩種方法：

- 刪除現有的 [GitHub 存放區](#)
- 建立其他名稱的存放區，並記得在下列指令中變更。

9. 將 `codelab-genai` 存放區新增為遠端 `origin`：

```
git remote add origin https://github.com/${GITHUB_USERNAME}/codelab-genai
```

9. 分享代碼

1. 確認您位於正確的目錄：

```
cd ~/codelab-genai
```

2. 將目前目錄中的所有檔案新增至這個提交：

```
git add .
```

3. 建立第一個修訂版本：

```
git commit -m "add http server"
```

4. 將提交內容推送至 `origin` 存放區的 `main` 分支版本：

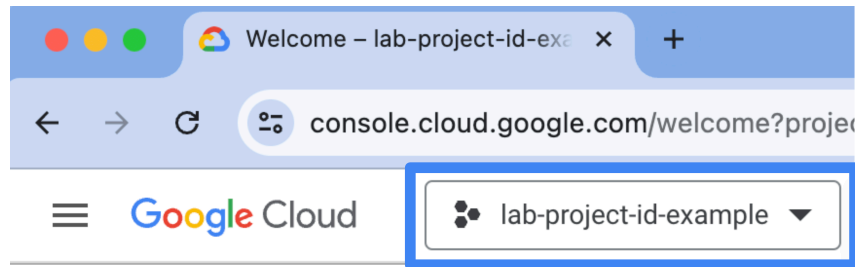
```
git push -u origin main
```

您可以執行這項指令，並前往產生的網址，在 GitHub 上查看應用程式程式碼：

```
echo -e "\n\nTo see your code, visit this URL:\n \n\n  https://github.com/${GITHUB_USERNAME}/codelab-genai/blob/main/src/app/app.component.ts\n\n\n"
```

10. 設定自動部署

1. 請讓 Cloud Shell 編輯器分頁保持開啟。稍後會再返回這個分頁。
2. 在新分頁中前往 [Cloud Run 頁面](#)
3. 在控制台中選取正確的 Google Cloud 專案



4. 按一下「CONNECT REPO」(連結存放區)
5. 按一下「使用 Cloud Build 設定」
 1. 選取「GitHub」做為「存放區供應商」
 - 如果尚未在瀏覽器中登入 GitHub 帳戶，請使用憑證登入。
 2. 依序點選「驗證」和「繼續」。
 3. 登入後，Cloud Run 頁面會顯示「您的所有存放區均未安裝 GitHub 應用程式」訊息。
 4. 按一下「INSTALL GOOGLE CLOUD BUILD」按鈕。
 - 在「安裝設定」頁面中，選取「Only select repositories」，然後選擇透過 CLI 建立的「codelab-genai」存放區。
 - 按一下「安裝」
 - 注意：如果 GitHub 存放區數量較多，載入作業可能需要幾分鐘的時間。
 5. 選取 `your-user-name/codelab-genai` 做為「存放區」
 - 如果沒有看到存放區，請按一下「管理已連結的存放區」連結。
 6. 將「Branch」(分支版本) 保留為 `^main$`
 7. 按一下「Go、Node.js、Python、Java、.NET Core、Ruby 或 PHP (透過 Google Cloud 的 Buildpacks 使用)」
 - 其他欄位 (`Build context directory`、`Entrypoint` 和 `Function target`) 則維持原狀。
 8. 按一下「儲存」。
6. 向下捲動至「驗證」
7. 按一下「允許未經驗證的叫用」
8. 按一下 [Create]

建構作業完成後 (需要幾分鐘)，請執行這項指令並前往產生的網址，查看正在執行的應用程式：

```
echo -e "\n\nOnce the build finishes, visit your live application:\n \n\n"
"$(\
  gcloud run services list | \
  grep codelab-genai | \
  awk '/URL/{print $2}' | \
  head -1 \
)
```


11. 變更驗證碼

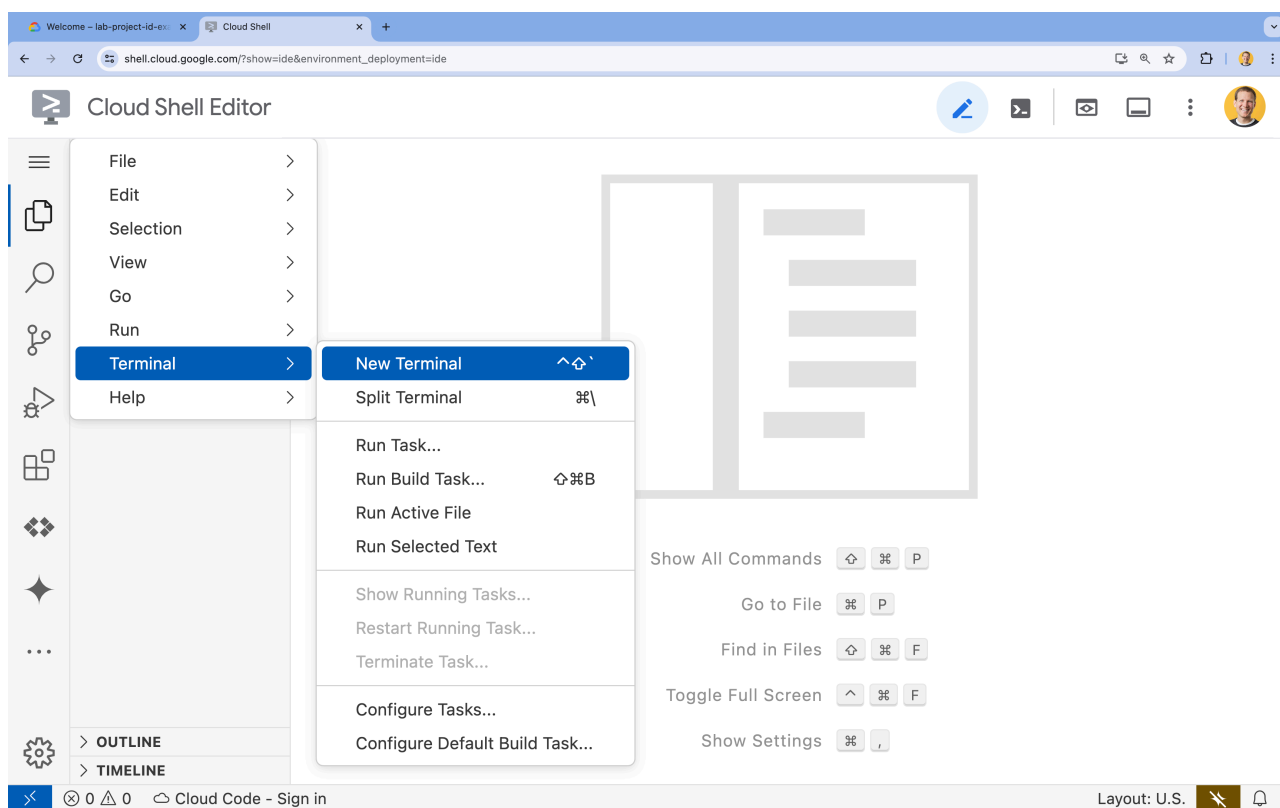
返回 Cloud Shell 編輯器

如果 Cloud Shell 編輯器仍處於開啟狀態，可以略過這些步驟。

1. 前往 [Cloud Shell 編輯器](#)
2. 如果畫面底部未顯示終端機，請開啟終端機：
 - 按一下漢堡選單



- 按一下「終端機」。
- 按一下「New Terminal」(新增終端機)



3. 在終端機中，使用下列指令設定專案：

- 格式：

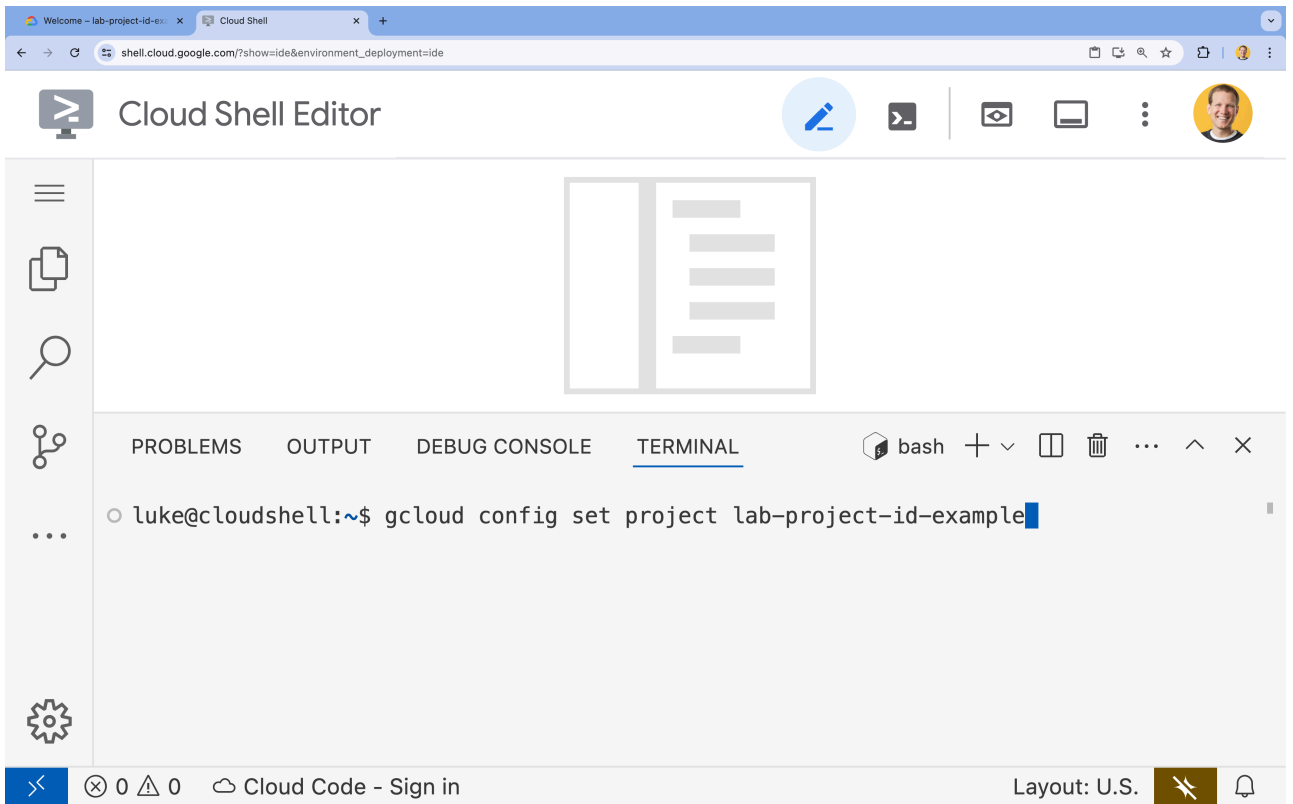
```
gcloud config set project [PROJECT_ID]
```

- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，請按照下列步驟操作：
 - 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



4. 如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/N)?` 提示，可能是輸入的專案 ID 有誤。按下 `N` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

在應用程式中新增 Vertex AI

1. 返回畫面底部的 Cloud Shell 終端機。
2. 確認您仍位於正確的目錄：

```
cd ~/codelab-genai
```

3. 安裝 Node.js Vertex AI SDK :

```
npm install @google-cloud/vertexai
```

4. 安裝 Node.js Google Auth SDK :

```
npm install google-auth-library
```

5. 在 Cloud Shell 編輯器中重新開啟 `server.ts`

```
cloudshell edit server.ts
```

6. 將 `server.ts` 檔案中的程式碼替換為：

```

// server.ts
import { APP_BASE_HREF } from '@angular/common';
import { CommonEngine } from '@angular/ssr';
import express from 'express';
import { fileURLToPath } from 'node:url';
import { dirname, join, resolve } from 'node:path';
import bootstrap from './src/main.server';

import { VertexAI } from '@google-cloud/vertexai';
import { GoogleAuth } from 'google-auth-library';

// The Express app is exported so that it can be used by serverless Functions.
export function app(): express.Express {
  const server = express();
  const serverDistFolder = dirname(fileURLToPath(import.meta.url));
  const browserDistFolder = resolve(serverDistFolder, '../browser');
  const indexHtml = join(serverDistFolder, 'index.server.html');
  const commonEngine = new CommonEngine();

  const auth = new GoogleAuth();

  server.set('view engine', 'html');
  server.set('views', browserDistFolder);

  // Example Express Rest API endpoints
  server.get('/api/facts', async (req, res) => {
    const project = await auth.getProjectId();

    const vertex = new VertexAI({ project: project });
    const generativeModel = vertex.getGenerativeModel({
      model: 'gemini-1.5-flash',
      generationConfig: { responseMimeType: 'application/json' }
    });

    const animal = req.query['animal'] || 'dog';
    const prompt = `Give me 10 fun facts about ${animal}.
      Return as json as an array in the format ['fact 1', 'fact 2']
      Remove backticks and other markdown formatting.`;

    const resp = await generativeModel.generateContent(prompt);
    let factArray = '';

    if (resp.response.candidates) {
      factArray = JSON.parse(resp.response.candidates[0].content.parts[0].text ||
    '');
    }
    res.send(factArray);
  });

  // Serve static files from /browser
  server.get('**', express.static(browserDistFolder, {

```

```

        maxAge: '1y',
        index: 'index.html',
    }));

    // All regular routes use the Angular engine
    server.get('**', (req, res, next) => {
        const { protocol, originalUrl, baseUrl, headers } = req;

        commonEngine
            .render({
                bootstrap,
                documentFilePath: indexHtml,
                url: `${protocol}://${headers.host}${originalUrl}`,
                publicPath: browserDistFolder,
                providers: [{ provide: APP_BASE_HREF, useValue: baseUrl }],
            })
            .then((html) => res.send(html))
            .catch((err) => next(err));
    });

    return server;
}

function run(): void {
    const port = process.env['PORT'] || 4000;

    // Start up the Node server
    const server = app();
    server.listen(port, () => {
        console.log(`Node Express server listening on http://localhost:${port}`);
    });
}

run();

```

7. 在 Cloud Shell 編輯器中開啟 `app.component.ts`

```
cloudshell edit src/app/app.component.ts
```

8. 將 `app.components.ts` 檔案中的程式碼替換為：

```
// app.component.ts
import { Component, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, FormsModule],
  template: `
    <section>
      <input
        type="text"
        placeholder="dog"
        [(ngModel)]="animal"
        class="text-black border-2 p-2 m-2 rounded"
      />
      <button
        (click)="getNewFunFacts()"
      >
        Get New Fun Facts
      </button>
      <ol>
        @for(fact of facts(); track fact) {
          <li>{{fact}}</li>
        } @empty {
          <li>No facts are available</li>
        }
      </ol>
    </section>
  `,
  styles: '',
})
export class AppComponent {
  animal = '';
  facts = signal<string[]>([]);

  getNewFunFacts() {
    fetch(`/api/facts?animal=${this.animal}`).then(response =>
    response.json()).then(facts => {
      this.facts.set(facts);
    });
  }
}
```

12. 重新部署

1. 確認您仍在 Cloud Shell 的正確目錄中：

```
cd ~/codelab-genai
```

2. 執行下列指令，將新版應用程式提交至本機 Git 存放區：

```
git add .  
git commit -m "add latest changes"
```

3. 將變更推送到 GitHub：

```
git push
```

4. 建構完成後，請執行下列指令並前往已部署的應用程式：

```
echo -e "\n\nOnce the build finishes, visit your live application:\n \  
$( \  
  gcloud run services list | \  
  grep codelab-genai | \  
  awk '/URL/{print $2}' | \  
  head -1 \  
)" \n\n"
```

建構作業可能需要幾分鐘才會完成，完成後您就能看到變更。

如要查看所有修訂版本的記錄，請前往：<https://console.cloud.google.com/run/detail/us-central1/codelab-genai/revisions>

13. (選用) 稽核 Vertex AI 使用情形

與其他 Google Cloud 服務一樣，您可以[稽核 Vertex AI 作業](#)。稽核記錄可協助您回答有關人事時地物的問題。Vertex AI 的管理稽核記錄預設為啟用。如要稽核內容生成要求，請[啟用資料存取稽核記錄](#)：

資料稽核記錄可能會產生額外費用。

1. 如果您是使用搜尋列尋找這個頁面，請選取子標題為「IAM & Admin」(IAM 與管理) 的結果。
2. 確認現有的 Google Cloud 專案是您建立 Cloud Run 應用程式的專案。
3. 在「資料存取稽核記錄設定」表格中，找出「服務」欄並選取 `Vertex AI API`。
4. 在「記錄類型」分頁中，選取「資料存取」稽核記錄類型 `Admin read` 和 `Data read`。
5. 按一下 [儲存]。

啟用後，您就能查看應用程式每次呼叫的稽核記錄。如要查看含有叫用詳細資料的稽核記錄，請按照下列步驟操作：

1. 返回已部署的應用程式，然後重新整理頁面以觸發記錄。
2. 前往 Google Cloud 控制台的「記錄檔探索工具」頁面：
3. 在查詢視窗中輸入：

```
LOG_ID("cloudaudit.googleapis.com%2Fdata_access")
protoPayload.serviceName="aiplatform.googleapis.com"
```

4. 點選「執行查詢」

稽核記錄會擷取 Vertex AI API 的使用情形，但無法觀察工作負載相關資料，例如提示或回應詳細資料。

14. (選用) 提高 AI 工作負載的可觀測性

稽核記錄不會擷取工作負載相關資訊，如要提高工作負載的可觀測性，您必須明確記錄這項資訊。您可以使用慣用的記錄架構來執行這項操作。以下步驟說明如何使用原生 Node.js 記錄機制執行這項操作。

1. 在 Cloud Shell 編輯器中重新開啟 `server.ts`

```
cloudshell edit ~/codelab-genai/server.ts
```

2. 呼叫 `await generativeModel.generateContent(prompt)` (第 19 行) 後，新增下列程式碼：

```
console.log(JSON.stringify({
  severity: 'DEBUG',
  message: 'Content is generated',
  prompt: prompt,
  response: resp.response,
}));
```

這段程式碼會使用[結構化記錄](#)格式，將生成內容的相關資訊寫入 `stdout`。Cloud Run 中的記錄代理程式會擷取列印至 `stdout` 和 [寫入](#)這個格式的輸出內容，並將其寫入 Cloud Logging。

3. 重新開啟 Cloud Shell，然後輸入下列指令，確認您位於正確的目錄：

```
cd ~/codelab-genai
```

4. 修訂變更：

```
git commit -am "observe generated content"
```

5. 將變更推送至 GitHub，觸發修改版本的重新部署作業：

```
git push
```

部署新版本後，您就能觀察對 Vertex AI 的呼叫相關偵錯資訊。

如要查看應用程式記錄，請按照下列步驟操作：

1. 前往 Google Cloud 控制台的「記錄檔探索工具」頁面：
2. 在查詢視窗中輸入：

```
LOG_ID("run.googleapis.com%2Fstdout")
severity=DEBUG
```

3. 點選「執行查詢」

查詢結果會顯示含有提示和 Vertex AI 回覆的記錄，包括可用於監控[安全做法](#)的「安全評分」

記錄提示時請務必謹慎，因為提示可能含有不應觀察的私密資訊。建議遮蓋或隱藏這類資訊。詳情請參閱 [DLP 說明文件](#)。

15. (選用) 清除

不使用服務時，Cloud Run 不會收費，但您可能仍須支付容器映像檔在 Artifact Registry 的儲存費用。您可以刪除 Cloud 專案，以免產生費用。刪除 Cloud 專案後，系統就會停止對專案使用的所有資源收取費用。

如要刪除專案，請按照下列步驟操作：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

您也可以從 Cloud Shell 磁碟刪除不必要的資源。您可以：

1. 刪除 Codelab 專案目錄：

```
rm -rf ~/codelab-genai
```

2. 警告！這項操作無法復原！如要刪除 Cloud Shell 中的所有內容來釋出空間，可以[刪除整個主目錄](#)。請務必將要保留的內容另存到其他位置。

```
sudo rm -rf $HOME
```

16. 恭喜

在本實驗室中，您編寫了網頁應用程式，並設定 [Cloud Run](#)，在應用程式的原始碼變更時自動部署應用程式。然後修改應用程式並重新部署。

如果你喜歡這項實驗室，可以嘗試使用其他程式設計語言或架構：

- [Python](#)
- [Node.js](#)
- [Go](#)
- [Java](#)
- [Angular](#)
- [Next.js](#)

如要參與使用者體驗研究，協助我們改善您今天使用的產品，請[按這裡註冊](#)。

以下提供幾種繼續學習的方式：

- 文件：使用 [Genkit](#) 做為彈性模型抽象化，輕鬆整合任何模型 API，並使用社群維護的模型。
 - 程式碼研究室：[如何在 Cloud Run 上部署 Gemini 支援的即時通訊應用程式](#)
 - [如何搭配使用 Gemini 函式呼叫與 Cloud Run](#)
 - [如何使用 Cloud Run Jobs Video Intelligence API 逐場景處理影片](#)
 - 隨選研討會：[Google Kubernetes Engine 上線](#)
-